Université Nationale des Sciences, Technologies,
Ingénierie et Mathématiques (UNSTIM)

Natitingou – Bénin

Licence 3 – Mathématiques et Informatique

Gestion d'un personnel universitaire

Héritage, polymorphisme et méthodes virtuelles en C++

Exercice 3 – Travail de groupe 11

Membres du groupe :

— CAPO-CHICHI Luce
— BANKOLE Grace
— DANDJINOU Alida
— ONIFADE Adio
— KINIFFO Mistourath
— BOURANDI Worou Alassane
— PADONOU Socrates

Enseignant : Dr CAPO-CHICHI Gilbert ...

Année universitaire 2026–2027

Table des matières

1 Introduction **2**

2 Analyse du problème **2**
2.1 Identification de la classe mère et des classes dérivées 2
2.2 Attributs et méthodes de chaque classe 2
2.3 Hypothèses de modélisation 2

3 Diagramme de classes UML **3**
3.1 Lecture du diagramme 3

4 Implémentation en C++ **3**
4.1 La classe abstraite `Employe` 3
4.2 Une classe dérivée : `Enseignant` 4
4.3 La collection polymorphe 4
4.4 Polymorphisme et liaison dynamique 4

5 Compilation et résultats **5**
5.1 Compilation et exécution 5
5.2 Résultat de l'exécution 5
5.3 Vérification des calculs 6

6 Conclusion **6**

1 Introduction

Une université emploie plusieurs catégories de personnel : des enseignants, des agents administratifs et des techniciens. Tous partagent des informations communes (matricule, nom, prénom, salaire de base), mais chaque catégorie perçoit des indemnités ou des primes qui lui sont propres. Le calcul du salaire diffère donc d'une catégorie à l'autre.

Ce travail a pour objectif de modéliser cette situation en programmation orientée objet. Il mobilise les notions suivantes : **héritage**, **classes abstraites**, **méthodes virtuelles**, **redéfinition**, **polymorphisme** et **liaison dynamique**.

Le rapport présente successivement l'analyse du problème, le diagramme de classes UML, l'implémentation en C++, puis les résultats de l'exécution.

2 Analyse du problème

2.1 Identification de la classe mère et des classes dérivées

Les trois catégories de personnel partagent les mêmes attributs de base. Pour éviter de répéter ce code, on les regroupe dans une classe mère.

— **Classe mère :** `Employe`, déclarée **abstraite**. On ne crée jamais un « employé » sans catégorie dans l'université.

— **Classes dérivées :** `Enseignant`, `AgentAdministratif` et `Technicien`.

Chaque classe dérivée *est un* `Employe` : la relation est donc une relation d'héritage public.

2.2 Attributs et méthodes de chaque classe

Classe	Attributs	Méthodes
`Employe` (abstraite)	`matricule`, `nom`, `prenom`, `salaireBase`	`calculerSalaire()` (virtuelle pure), `afficher()`, destructeur virtuel
`Enseignant`	`nbHeuresSup`, `tauxHeureSup`, `indemniteRecherche`	`calculerSalaire()`, `afficher()`
`AgentAdministratif`	`primeRendement`, `primeResponsabilite`	`calculerSalaire()`, `afficher()`
`Technicien`	`primeRisque`, `primeAstreinte`	`calculerSalaire()`, `afficher()`

TABLE 1 – Attributs et méthodes des classes

2.3 Hypothèses de modélisation

L'énoncé ne fixe pas les formules de calcul. Nous avons retenu les règles suivantes :

$$\begin{aligned}
\text{Salaire}_{\text{Enseignant}} &= \text{salaireBase} + \text{nbHeuresSup} \times \text{tauxHeureSup} + \text{indemniteRecherche} \\
\text{Salaire}_{\text{Agent}} &= \text{salaireBase} + \text{primeRendement} + \text{primeResponsabilite} \\
\text{Salaire}_{\text{Technicien}} &= \text{salaireBase} + \text{primeRisque} + \text{primeAstreinte}
\end{aligned}$$

Les montants sont exprimés en FCFA et les valeurs utilisées dans le programme sont des données de test.

3 Diagramme de classes UML

Le diagramme a été réalisé dans Visual Studio Code avec l'extension PlantUML.

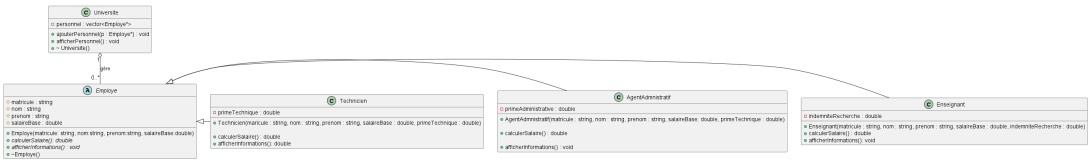


FIGURE 1 – Diagramme de classes UML de la gestion du personnel

3.1 Lecture du diagramme

— `Employe` est **abstraite** : son nom est en italique et sa méthode `calculerSalaire()` est virtuelle pure.
— Les trois flèches à **triangle vide** représentent la relation d'**héritage** ; le triangle pointe vers la classe mère.
— Les attributs de `Employe` sont `protected` (symbole `#`) afin que les classes dérivées puissent les utiliser.
— Les attributs propres à chaque classe dérivée sont `private` (symbole `-`).

4 Implémentation en C++

Le programme complet est fourni dans le fichier `src/main.cpp` du dépôt GitHub. Cette section en commente les parties essentielles.

4.1 La classe abstraite `Employe`

```
1  class Employe {
2  protected:
3      string matricule, nom, prenom;
4      double salaireBase;
5  public:
6      Employe(const string& matricule, const string& nom,
7              const string& prenom, double salaireBase);
8
9      virtual ~Employe() = default;                    // destructeur
       virtuel
10     virtual double calculerSalaire() const = 0;     // virtuelle pure
```

```
    virtual void afficher() const;                    // virtuelle
};
```

— `= 0` rend la méthode **virtuelle pure** : la classe devient abstraite et ne peut plus être instanciée.
— Le **destructeur virtuel** garantit que la destruction d'un objet dérivé à travers un pointeur `Employe*` appelle bien le bon destructeur.

4.2 Une classe dérivée : `Enseignant`

```
class Enseignant : public Employe {
private:
    int nbHeuresSup;
    double tauxHeureSup, indemniteRecherche;
public:
    double calculerSalaire() const override {
        return salaireBase + nbHeuresSup * tauxHeureSup +
            indemniteRecherche;
    }
    void afficher() const override {
        cout << "[ENSEIGNANT]\n";
        Employe::afficher();      // réutilise l'affichage de la classe
            mère
        // ... affichage des informations propres à l'enseignant
    }
};
```

Le mot-clé `override` demande au compilateur de vérifier que la méthode redéfinit bien une méthode virtuelle de la classe mère. Les classes `AgentAdministratif` et `Technicien` suivent le même schéma avec leurs propres formules.

4.3 La collection polymorphe

```
vector<unique_ptr<Employe>> personnel;

personnel.push_back(make_unique<Enseignant>(
    "ENS001", "ADJOVI", "Marc", 450000, 20, 5000, 75000));
personnel.push_back(make_unique<AgentAdministratif>(
    "ADM001", "KPADE", "Julien", 300000, 40000, 25000));
personnel.push_back(make_unique<Technicien>(
    "TEC001", "AGBO", "Ibrahim", 280000, 30000, 20000));

for (const auto& e : personnel) {
    e->afficher();                                // liaison dynamique
    masseSalariale += e->calculerSalaire();       // liaison dynamique
}
```

Un seul `vector` contient des objets de trois types différents, tous manipulés via le type `Employe`.

4.4 Polymorphisme et liaison dynamique

Dans la boucle, le type de `e` est `Employe`. Pourtant, l'appel `e->calculerSalaire()` exécute la version de `Enseignant`, `AgentAdministratif` ou `Technicien` selon le **type**

réel de l'objet. Ce choix est fait **à l'exécution** grâce à la table des méthodes virtuelles : c'est la **liaison dynamique**. Le même code produit donc des comportements différents : c'est le **polymorphisme**.

5 Compilation et résultats

5.1 Compilation et exécution

```
g++ -std=c++17 -Wall -Wextra -o personnel main.cpp
./personnel
```

La compilation s'effectue sans erreur ni avertissement.

5.2 Résultat de l'exécution

Après compilation et lancement du programme, les informations des six employés sont affichées. Chaque salaire est calculé en appliquant la formule propre à la catégorie de l'employé. La sortie ci-dessous présente les résultats de manière synthétique.

Listing 1 – Résultat de l'exécution du programme

```
========== GESTION DU PERSONNEL UNIVERSITAIRE ==========

[ENSEIGNANT]
Nom et prénom : ADJOVI Marc
Matricule : ENS001
Salaire : 625000 FCFA

[ENSEIGNANTE]
Nom et prénom : HOUNKPE Sylvie
Matricule : ENS002
Salaire : 682000 FCFA

[AGENT ADMINISTRATIF]
Nom et prénom : KPADE Julien
Matricule : ADM001
Salaire : 365000 FCFA

[AGENT ADMINISTRATIF]
Nom et prénom : DOSSOU Awa
Matricule : ADM002
Salaire : 385000 FCFA

[TECHNICIEN]
Nom et prénom : AGBO Ibrahim
Matricule : TEC001
Salaire : 330000 FCFA

[TECHNICIENNE]
Nom et prénom : SANNI Grace
Matricule : TEC002
Salaire : 330000 FCFA

-----------------------------------------------------
Masse salariale totale : 2717000 FCFA
=====================================================
```

La masse salariale totale correspond à la somme des salaires des six employés :

$$625\,000 + 682\,000 + 365\,000 + 385\,000 + 330\,000 + 330\,000 = 2\,717\,000 \text{ FCFA}.$$

Cette exécution illustre le polymorphisme : les employés sont traités à travers le type commun `Employe`, mais le calcul du salaire dépend de la classe réelle de chaque objet.

5.3 Vérification des calculs

Employé	**Calcul**	**Salaire (FCFA)**
ADJOVI Marc (Enseignant)	$450\,000 + 20 \times 5\,000 + 75\,000$	625 000
HOUNKPE Sylvie (Enseignante)	$520\,000 + 12 \times 6\,000 + 90\,000$	682 000
KPADE Julien (Agent)	$300\,000 + 40\,000 + 25\,000$	365 000
DOSSOU Awa (Agent)	$320\,000 + 35\,000 + 30\,000$	385 000
AGBO Ibrahim (Technicien)	$280\,000 + 30\,000 + 20\,000$	330 000
SANNI Grace (Technicienne)	$290\,000 + 25\,000 + 15\,000$	330 000

TABLE 2 – Vérification manuelle des salaires affichés

La masse salariale totale est donc de **2 717 000 FCFA**.

6 Conclusion

Ce travail a permis de modéliser un personnel universitaire hétérogène grâce à l'héritage. La classe abstraite `Employe` factorise ce qui est commun, et la méthode virtuelle pure `calculerSalaire()` impose à chaque catégorie sa propre règle de calcul. Grâce au polymorphisme, une seule collection et une seule boucle suffisent pour traiter tous les employés.

L'ajout d'une nouvelle catégorie (par exemple un vacataire) ne demande que la création d'une nouvelle classe dérivée, sans modifier le code existant.